Algorithms Analysis

- ✓ Sequential, random, binary search.
- ✓ Bubble sort, Mergesort, quicksort.
- ✓ A common technique: Divide and conquer

- Solving recurrences
- Strassen's matrix multiplication
- Closest-pair and Convex-Hull problems

Searching Algorithm

Randomized searching in an unsorted array.

- Loop forever
  - Guess a number m from 1 to N. (or, 0 to N-1)
  - If A[m] is the target, return m.

- If the target exists, what is the expected #iteration?
- Expected #iteration=
- (1/n)(1) + ((n-1)/n)(1/n)(2)+ ((n-1)/n)((n-1)/n)(1/n)(3)+ ((n-1)/n)((n-1)/n)((n-1)/n)(1/n)(4)+ …

- Let $x = (n-1)/n < 1$
- Expected #iteration
- $=(1/n)(1) + ((n-1)/n)(1/n)(2)+ ((n-1)/n)((n-1)/n)(1/n)(3)+ ((n-1)/n)((n-1)/n)((n-1)/n)(1/n)(4)+ \ldots$
- $=(1/n)[1x^0+2x^1+3x^2+4x^3+\ldots \ ]$
- $=(1/n)(1/(1-x)^2)$
- $=n$
- Expected time=O(n)
- Worst case time= infinity
- Best case time=O(1)

It is just an example of randomized algorithm. Practically, we have no reason to use this for searching arrays.

Average case of Sequential search (for linked list, and unsorted array) — O(n)

- If the target is in the first position→1 step
- If the target is in the 2nd position→ 2 steps
- If the ….. kth positions → k steps

- Every position has the same probability – 1/n.
- Expected time=(1/n)(1)+(1/n)(2)+ … +(1/n)(n)=(1/n)(1+2+…+n)=(n+1)/2=O(n)

Average case of Sequential search (for linked list, and unsorted array) — O(n)

- From the view of conditional probability:

- 1 step, prob=1/n
- 2 steps, prob=[(n-1)/n] [1/(n-1)]=1/n
- 3 steps, prob=[(n-1)/n] [(n-2)/(n-1)][1/(n-2)]=1/n
- ……
- Expected time=(1/n)(1)+(1/n)(2)+ … +(1/n)(n)=(1/n)(1+2+…+n)=(n+1)/2=O(n)

Searching-Revisit

- For a sorted or unsorted linked list,
  - O(n) time.
- For a sorted array,
  - O(log n) time, binary search is used.
- For an unsorted array,
  - O(n) time.
- Can we do better in each case?

For sorted arrays

- Is O(log n) time the optimal worst case?
  - For comparison-based algorithms, yes.
  - O(log n) is also the lower bound of worst case.
  - We cannot do better.
  - Decision tree.

For linked list, sorted or unsorted

- Is O(n) the optimal worst case?
- Yes, because
- O(n) is the lower bound of worst case.
- Proof: A worst case scenario: the target is in the middle. The time to reach it from each end is n/2 = O(n). Q.E.D.
- Note,
- **lower bound of worst case ≠ best case.**

For unsorted array with arbitrary order.

- Is O(n) the optimal worst case?
- Yes.
- The technique used to prove this is called “fooling algorithm”.
- It is not an algorithm, but a technique for proving.

Average case analysis for upper bounds

- For a sorted or unsorted linked list,
  - O(n) time.
- For a sorted array,
  - O(log n) time, binary search is used.
- For an unsorted array,
  - O(n) time.
- All of them are the same as worst case.

Techniques in probability will be used.

Average case time complexity for binary search

- In a sorted array.
- If the target is at the middle, it requires 1 step.
- One element requires 1 step.
- Two elements require 2 steps.
- Four elements require 3 steps.
- Eight elements require 4 steps.
- …
- 2^{k-1} elements require k steps.
- …
- 2^{m-2} elements require m-1 steps
- $<=2^{m-1}$ elements require m steps

- For big-O notation, it is enough to consider this only.
- Why?

Average case time complexity $\Omega(\log n)$

- $1+2+4+\ldots+2^{m-2} < n$
- $1+2+4+\ldots+2^{m-2} + 2^{m-1} >= n$
- Therefore, $2^{m-1} -1 < n <= 2^{m} -1$
- $\log(n+1) <= m < \log(n+1)+1$

- Recall that 2^{m-2} elements require m-1 steps.
- → >= (n+1)/4 elements require >= log(n+1)-1 steps.

- Totally, there are more than (¼)(n+1)[log(n+1)-1] steps.
- On average, each element requires more than (1/n)(¼)(n+1)[log(n+1) -1] steps.
- → $\Omega(\log n)$

Average case time complexity $\Theta(\log n)$

- Average case time complexity
- < worst case time complexity
- = O(log n)
- Therefore,
- average case time complexity= $\Theta(\log n)$

Sorting Algorithms

Sorting—
Design and Analysis of Algorithms

- Standard algorithms
  - Bubble sort
  - Selection sort
  - Insertion sort
  - Mergesort (not in place)
  - Quicksort
  - Heapsort (easy, but not for this course)

- We try to learn the way to prove the correctness and the time complexity.

- Suppose the required outputs are in ascending order.

Selection sort
Another example of correctness proof

- S=the input set of elements of size N.
- While S is not empty
  - Take out the smallest one from S, and output it.

- After each iteration, the output sequence is in order, and if there are remaining elements, then, the last element in the output sequence is no more than all remaining elements.
- We use induction on m, the number of iterations passed.
- Base case: m=1. After the first iteration, the smallest element x has been taken away to the output. It is no more than any element left in S. The output sequence has only one element, and therefore, in order. The statement is true for m=1.

Selection sort
Another example of correctness proof

- Assume the statement is true for m=k.
- Consider the statement for m=k+1.
- Take out the smallest element x from S. Then, x is no more than any remaining element.
- Let the last element in the output be y. By inductive assumption, y<=x.
- Append x to the output sequence. Then, the output sequence is in order, and x becomes the last element which is no more than any element in S. Done!
- Result follows from induction.

Design and Analyze together.

- When we are writing recursions in algorithms/programming, we are doing mathematical induction in our mind.

- Mergesort and Quicksort are examples.

Mergesort

- Suppose that a function merge(L1, L2) which accepts two sorted linked lists, L1 and L2, merges them into one sorted linked list, L, and outputs L.
- merge(L1, L2) needs O(|L1|+|L2|) time.

Mergesort(L)

- Given a unsorted linked list, L.
- If |L|=0 or 1, return L.

- Statement: Mergesort(L) sorts the linked list L.
- Base cases: if |L|=0 or |L|=1, L is sorted already. The statement is true for |L|=0 or 1.

Mergesort(L)

- Divide the linked list into two halves, L1 and L2.
- Call Mergesort(L1);
- Call Mergesort(L2);
- L=Merge(L1,L2);
- Return L;

- Apply induction assumption to the two linked list, L1 and L2.
- That is, Mergesort(L1) sorts the linked list L1, and Mergesort(L2) sorts the linked list L2.
- L=Merge(L1,L2) merges L1 and L2 together to L.

Mergesort(L)

- What is the difficulty when you program Mergesort(L)?

- The function --- Merge(L1,L2) is the burden!

A common technique: Divide and Conquer

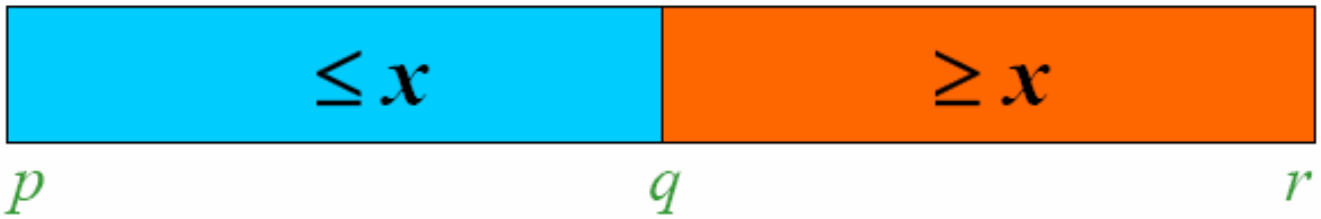
- Note, mergesort is a divide and conquer algorithm.
- Binary search is also divide and conquer.
- The coming quicksort is, too.

Quicksort

- Sort-in-place, like bubble sort and heapsort.
- Faster than mergesort, in practice
- For array.

Quicksort

1. Divide: Partition the array into 2 subarrays such that elements in the lower part ≤ elements in the higher part

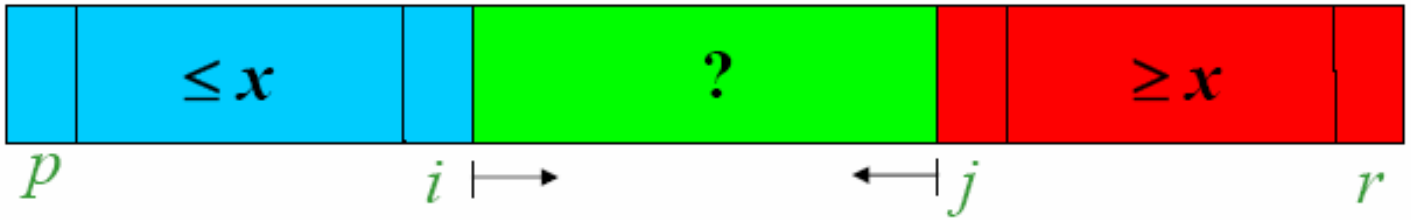


2. Conquer: Recursively sort 2 subarrays
3. Combine: Trivial (because in-place)

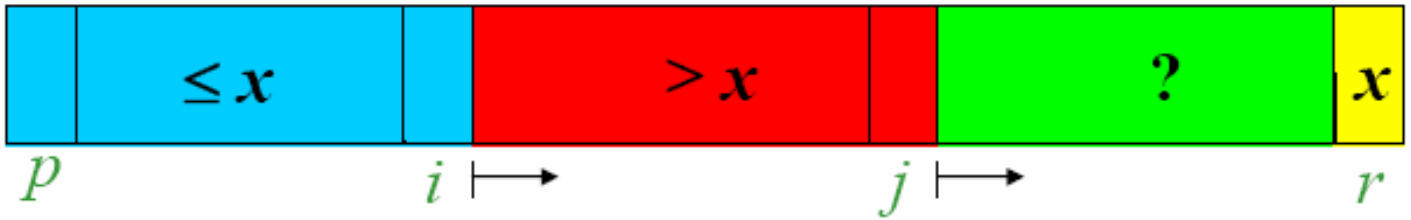
- Key: Linear-time ($\Theta(n)$) partitioning algorithm

Two partitioning algorithms

1. Hoare's algorithm: Partitions around the first element of subarray ($pivot = x = \mathrm{A}[p]$)



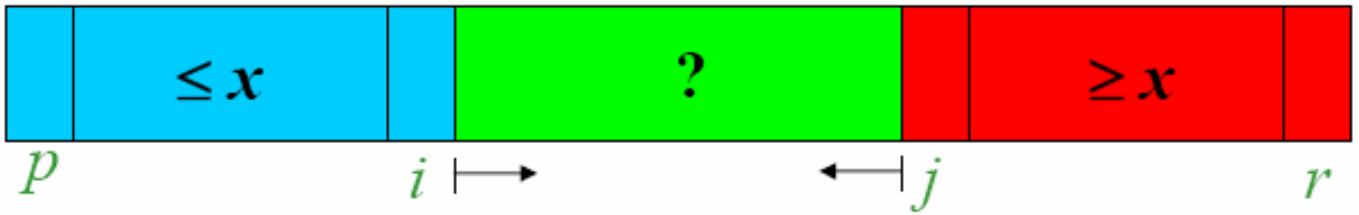
2. Lomuto's algorithm: Partitions around the last element of subarray ($pivot = x = \mathrm{A}[r]$)



Hoare's Partitioning Algorithm

```
H-PARTITION (A, p, r)
    pivot ← A[p]
    i ← p − 1
    j ← r + 1
    while true do
        repeat j ← j − 1 until A[j] ≤ pivot
        repeat i ← i + 1 until A[i] ≥ pivot
        if i < j then
            exchange A[i] ↔ A[j]
        else
            return j
```

Running time is $O(n)$

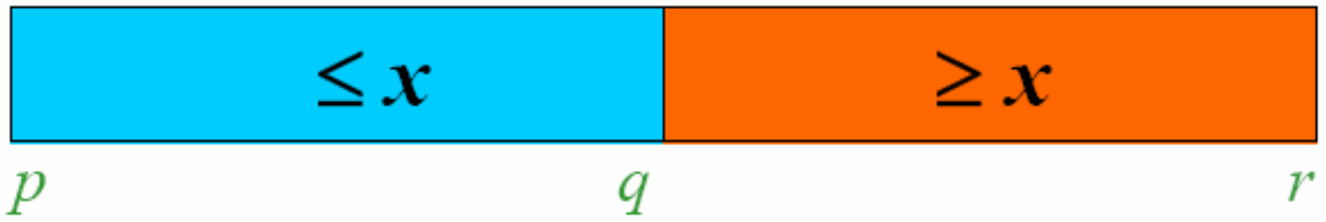


If p>=r then return

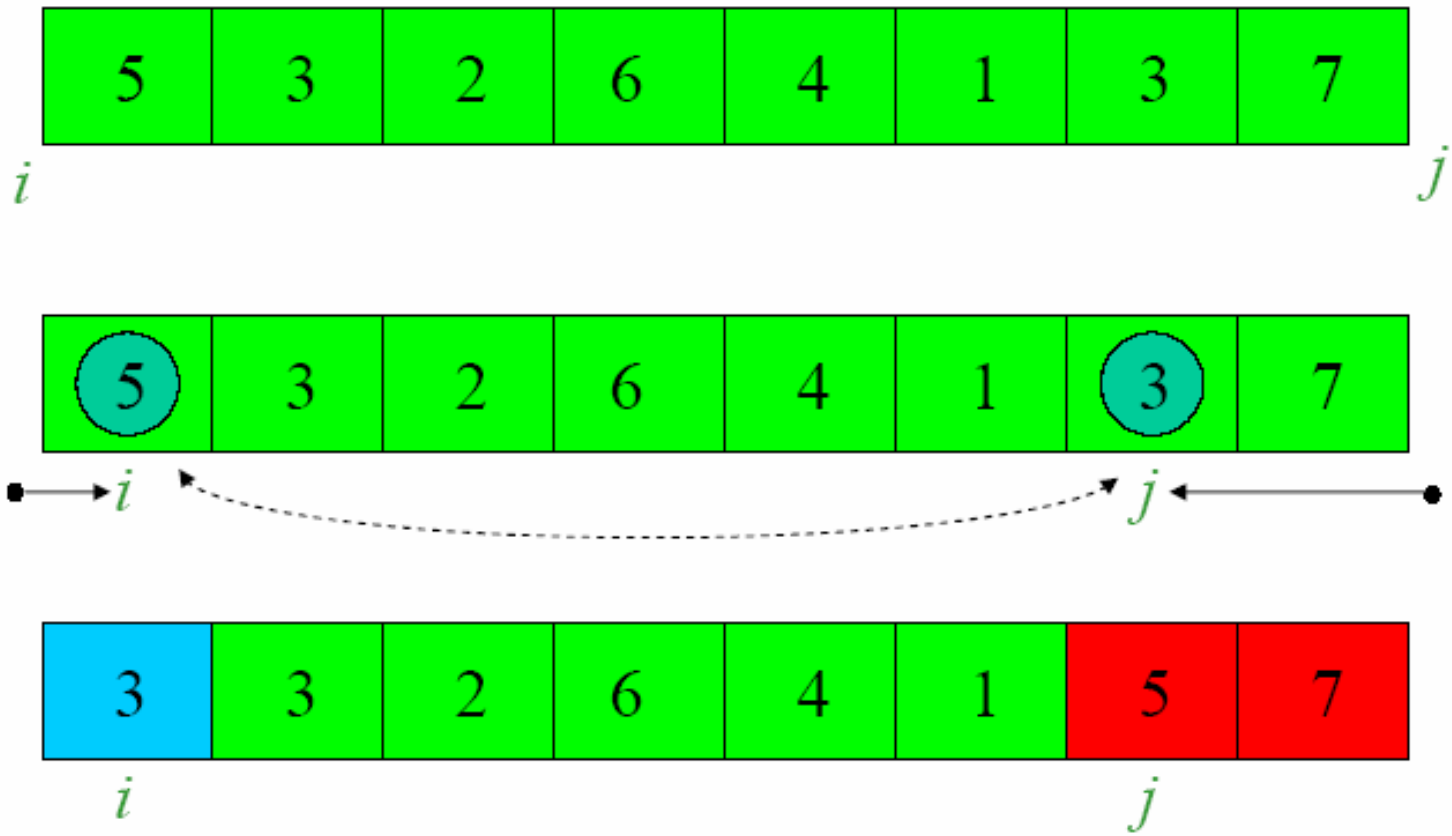
```
QUICKSORT (A, p, r)
    if p < r then
        q ← H-PARTITION(A, p, r)
        QUICKSORT(A, p, q)
        QUICKSORT(A, q +1, r)
```

Note, the burden of writing this algorithm is at partitioning.

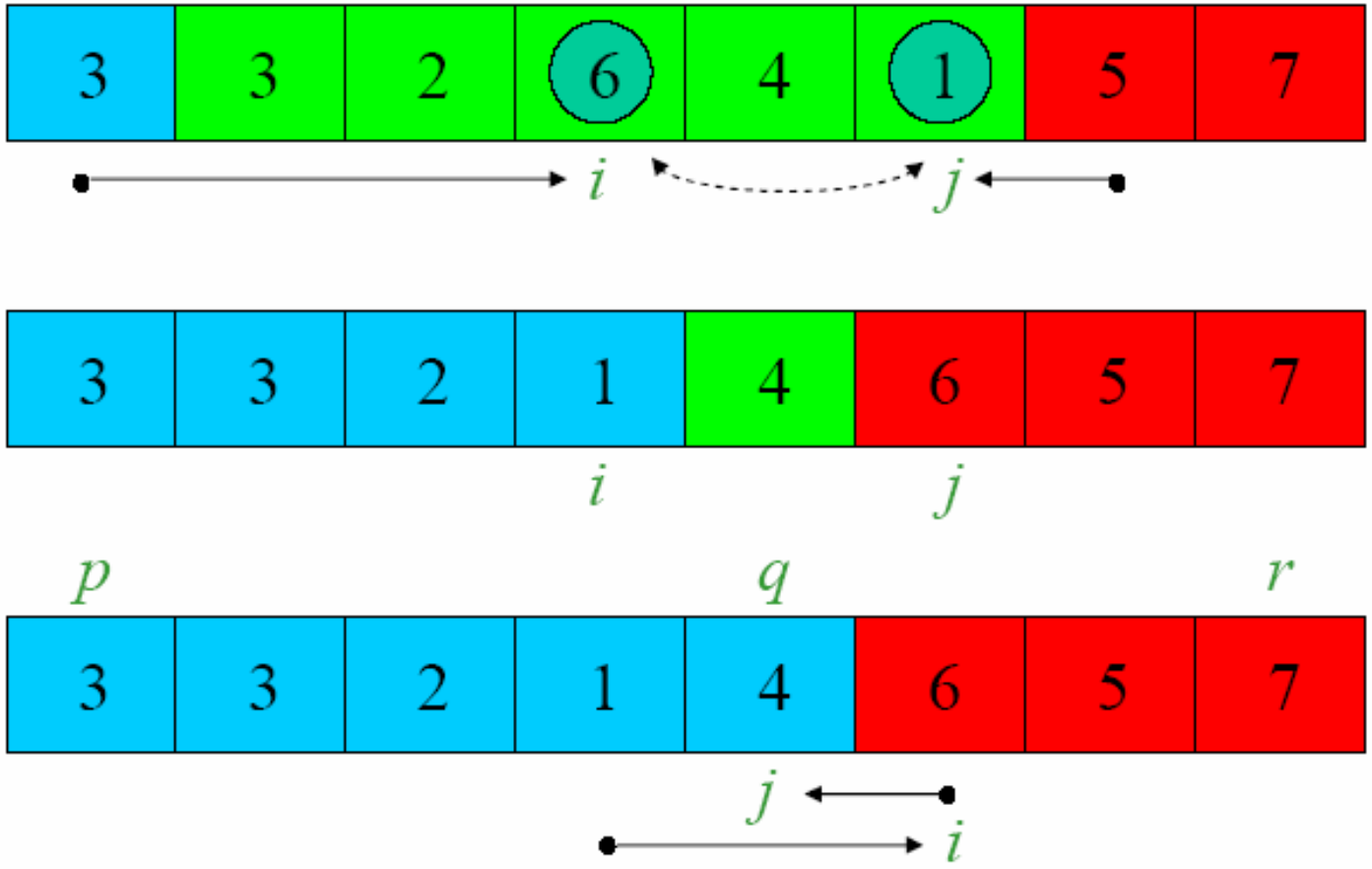
Initial invocation: QUICKSORT(A, 1, n)



Hoare's Algorithm: Example 1 (pivot = 5)

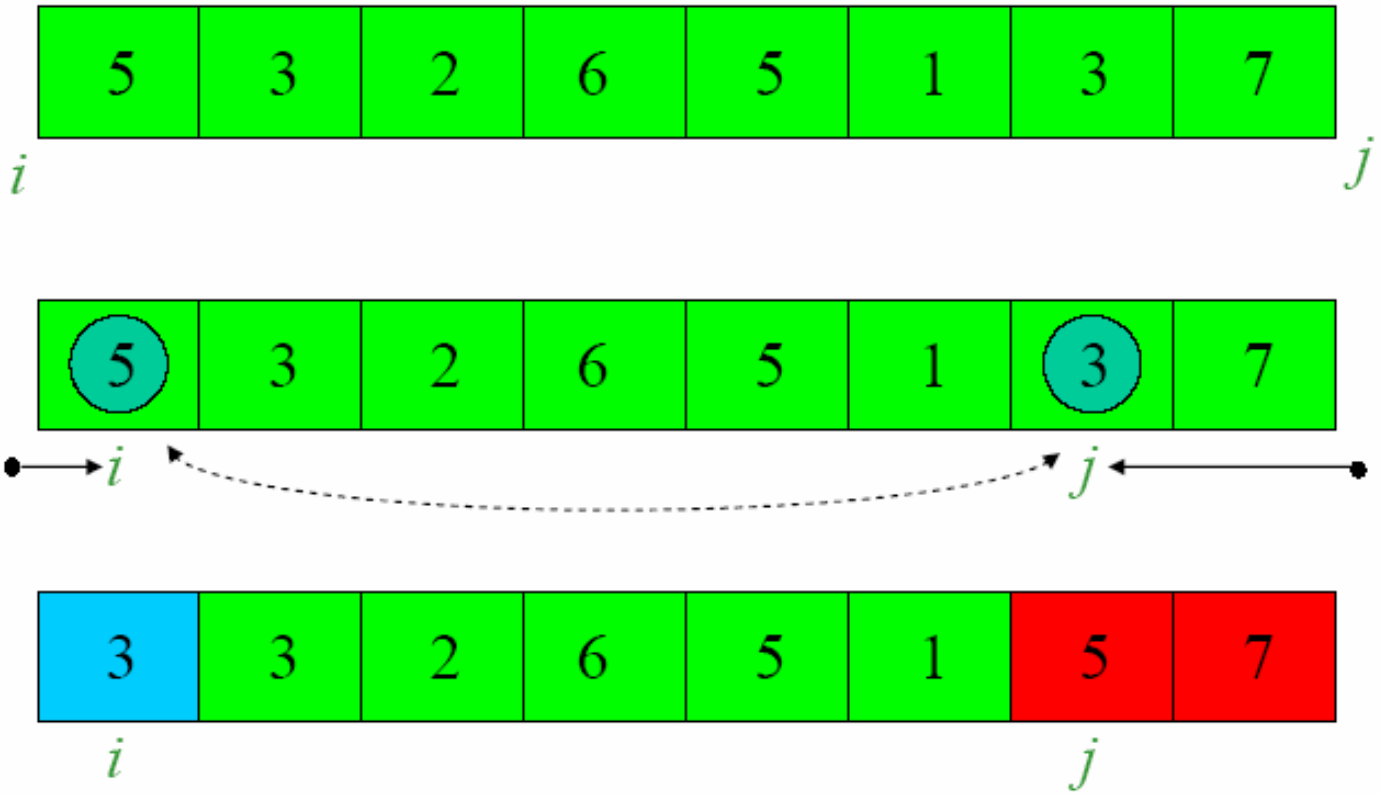


Hoare's Algorithm: Example 1 (pivot = 5)

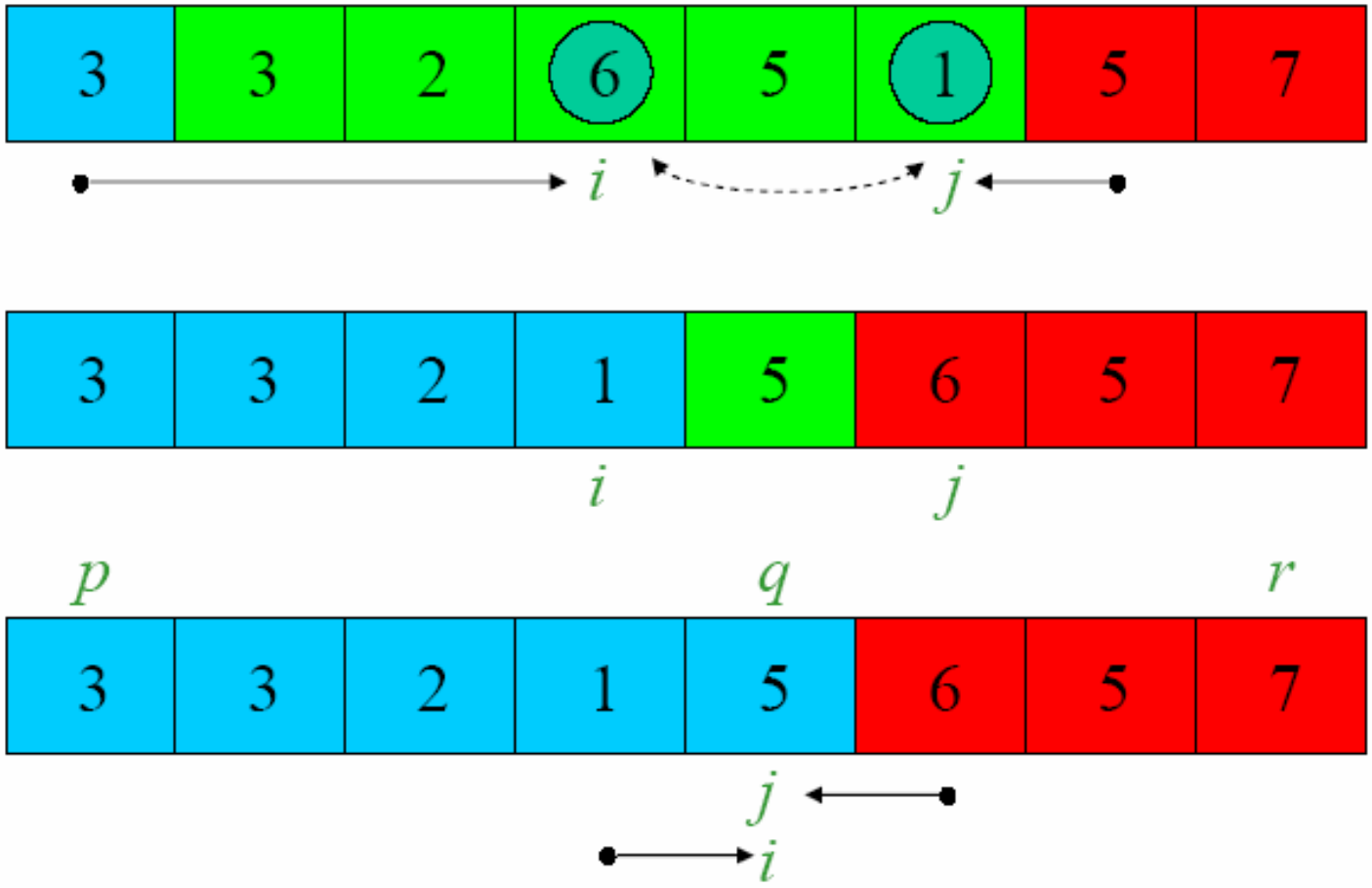


Termination: $i = 6; j = 5,$ i.e., $i = j + 1$

Hoare's Algorithm: Example 2 (pivot = 5)



Hoare's Algorithm: Example 2 (pivot = 5)



Termination: $i = j = 5$

Why O(n)-time?

#step is finite

- O(1)
- O(1)
- O(1)
- O(n)
- O(n)
- O(n)
- O(1)
- O(1)
- O(1)
- O(1)

```
H-PARTITION (A, p, r)
    pivot ← A[p]
    i ← p − 1
    j ← r + 1
    while true do
        repeat j ← j − 1 until A[j] ≤ pivot
        repeat i ← i + 1 until A[i] ≥ pivot
        if i < j then
            exchange A[i] ↔ A[j]
        else
            return j
```

Running time is $O(n)$